Vortexic Virtual Beam (VVB)

AND

Phase–Channel Allocator (PCA): A Geometric Transport Model for Smooth, Multi-Channel Flow

Rachel Alderman J. Scott Graber, PS Lumen Virtus

October 21, 2025

Abstract

We present a simple geometric transport model, the *Vortexic Virtual Beam* (VVB), and an allocation policy, the *Phase–Channel Allocator* (PCA), that together reduce "glugging" at bottlenecks by distributing flows across multiple channels and enabling low-cost migrations at preferred crossover elevations ("Kiss heights"). We formalize the address space (c, ϕ, z), derive the projected tangency condition for channel pairs, and show a lightweight policy that preserves ordering while responding to contention.

Vortexic section (core functional). We summarize the cross–sectional interaction of packets by

$$Z_0 \;=\; \lim_{\Delta \to 0} \sum_{i=1}^{n} \frac{\vec{P}_i \cdot \vec{R}_i}{D_i},$$

which returns a scalar axial reference Z_0 indicating where the channels most strongly couple (the "kiss" height).

Terms. $\vec{P}_i$ is the phase-direction unit vector of packet i in the section (e.g., $(\cos\phi_i, \sin\phi_i)$); $\vec{R}_i$ is the radial vector from the channel center to packet i (with magnitude $r(z_i)$); D_i is a local separation metric (phase/axial gap; see Methods) that is small near conflict, so nearby, well-aligned packets contribute more.

Intuition. The dot product $\vec{P}_i \cdot \vec{R}_i$ measures how strongly packet i's phase aligns with the channel's radial structure; dividing by D_i weights toward imminent interactions. As $\Delta \to 0$ the sum approaches a continuous section functional whose maximizer marks a preferred axial alignment.

How we use it here. This functional motivates the existence of discrete "kiss rungs" spaced by $\Delta z = \pi/k$ in our geometry. In the simulator, the PCA policy implements this idea discretely: it biases hops toward the nearest kiss rung (`prefer_kiss`) and uses the phase-smooth step $\Delta z = \pi/k$. We do not evaluate the full sum each frame; we use it as the continuous rationale behind the discrete rules tested in the sweeps and ablations.

1 Field and Coordinates

Let channel centers along X be $\{X_c\}$ and define the radius profile

$$r(z) = R_0\, e^{-\beta z}. \tag{1}$$

The phase field around channel X_c is

$$\phi(x, y, z; X_c) = \mathrm{atan2}(y,\, x - X_c) + kz. \tag{2}$$

Forward map $(c, \phi, z) \mapsto (x, y, z)$:

$$x = X_c + r(z)\cos(\phi - kz), \quad y = r(z)\sin(\phi - kz). \tag{3}$$

Round-trip check: $\phi' = \mathrm{atan2}(y, x - X_c) + kz = \phi$.

2 Projected Tangency and Kiss Height

For neighboring channels (X_a, X_b) with $X_a < X_b$, the x–z branches on $y = 0$ are

$$x_a(z) = X_a + r(z), \qquad x_b(z) = X_b - r(z). \tag{4}$$

Projected tangency ("Kiss") occurs when $x_a(z) = x_b(z)$, i.e. $r(z) = \frac{\Delta X}{2}$ with $\Delta X = X_b - X_a$. For $r(z) = R_0 e^{-\beta z}$,

$$z_{\mathrm{kiss}} = \frac{1}{\beta} \ln\!\left(\frac{2R_0}{\Delta X}\right), \qquad \Delta X \le 2R_0. \tag{5}$$

Phase-continuous hop between $\phi = 0$ and $\phi = \pi$ branches uses the vertical offset

$$\Delta z_\phi = \frac{\pi}{k}. \tag{6}$$

3 Equal-Weight Separatrix

With symmetric gains $s_a(z) = s_b(z)$ the equal-weight crossover on $y = 0$ is the midpoint

$$x_{\mathrm{eq}}(z) = \frac{x_a(z) + x_b(z)}{2}. \tag{7}$$

In the general case $x_{\mathrm{eq}}(z) = \frac{s_a x_b + s_b x_a}{s_a + s_b}$.

4 Policy: Phase–Channel Allocator (PCA)

We detect same-channel conflicts by thresholds on phase and height:

$$|\Delta\phi| < \phi_{\min} \quad \wedge \quad |\Delta z| < z_{\min}. \tag{8}$$

On conflict and outside a hysteresis window, choose a target channel with minimum local density near (ϕ, z), bias toward z_{kiss} of the neighbor pair, apply the phase-smooth offset Δz_ϕ, nudge ϕ by a small golden-angle to avoid immediate re-collision, and switch channels. Ordering is preserved per-flow via sequence numbers or reserved ϕ windows.

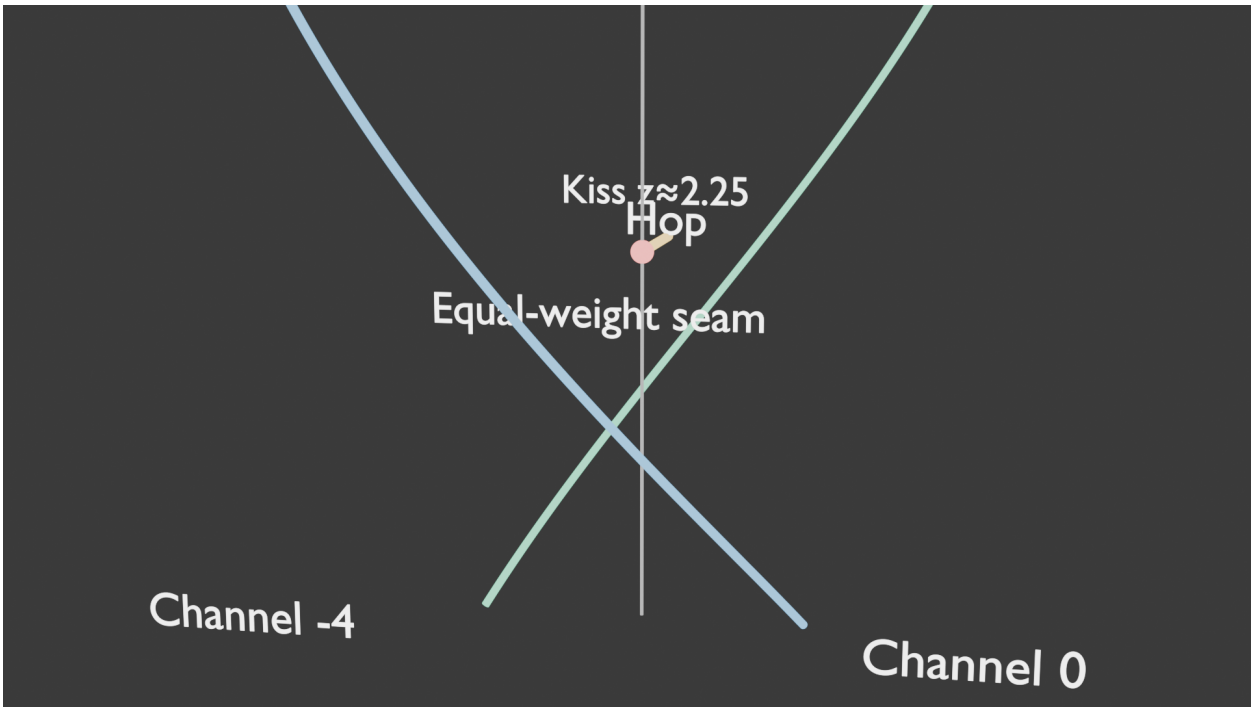


Figure 1: Annotated Vortexic Virtual Beam (VVB) geometry. Light blue and green curves show constant-phase streamlines around the neighbor channels at $x = -4$ and $x = 0$. The vertical gray line is the equal-weight seam at $x = -2$. The pink dot marks the *Kiss* height $z_{\text{kiss}} = \frac{1}{\beta} \ln\left(\frac{2R_0}{\Delta x}\right)$, which for R_0=3.0, β=0.18, and Δx=4 evaluates to $z_{\text{kiss}} \approx 2.25$. The orange arc depicts a hop across the seam at z_{kiss}.

5 Metrics

We report:

- **Hop count** and hop matrix by channel pair.
- **Glugging index** = mean coefficient of variation of per-channel occupancy across frames.
- Histograms of **hop height** z and distance to the Kiss seam $|z - z_{\text{kiss}}|$.

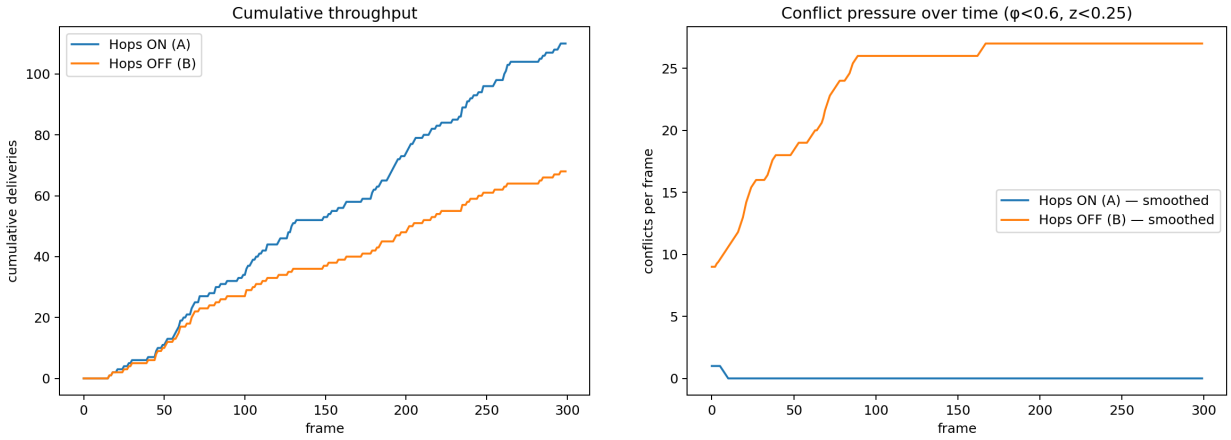
6 Experimental Setup

Unless stated otherwise, we use R_0=3.0, β=0.18, k=0.35; channels at $[-4, 0, 4]$; N packets uniformly initialized; thresholds $\phi_{\min}, z_{\min}$; and hysteresis H. The demo script streams NDJSON frames with packet states and hop events.

7 Results

Condition	Throughput (deliveries/frame) ↑	Gain	Notes
Hops ON	0.367	**+61.7%**	stalls-on-conflict model
Hops OFF	0.227	–	identical seed/params

Table 1: Under a stall-on-conflict service model, hopping increases delivered throughput by ∼62%.



(a) Cumulative deliveries (A above B).

(b) Conflicts per frame (A $\ll$ B).

Figure 2: A/B comparison with stalls-on-conflict: hopping increases delivered throughput and suppresses conflict pressure.

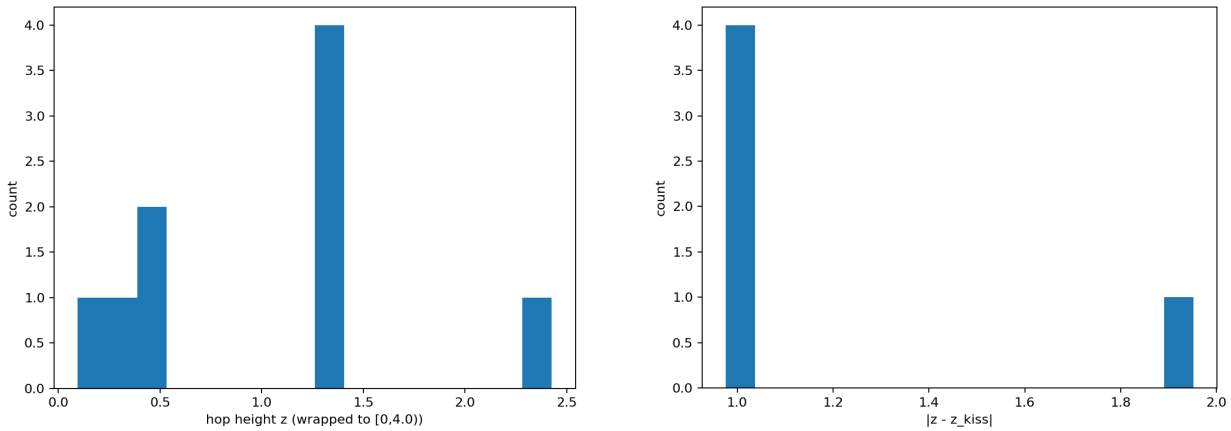


Figure 3: Left: distribution of hop heights z. Right: $|z - z_{\text{kiss}}|$; concentration near 0 shows kiss-bias.

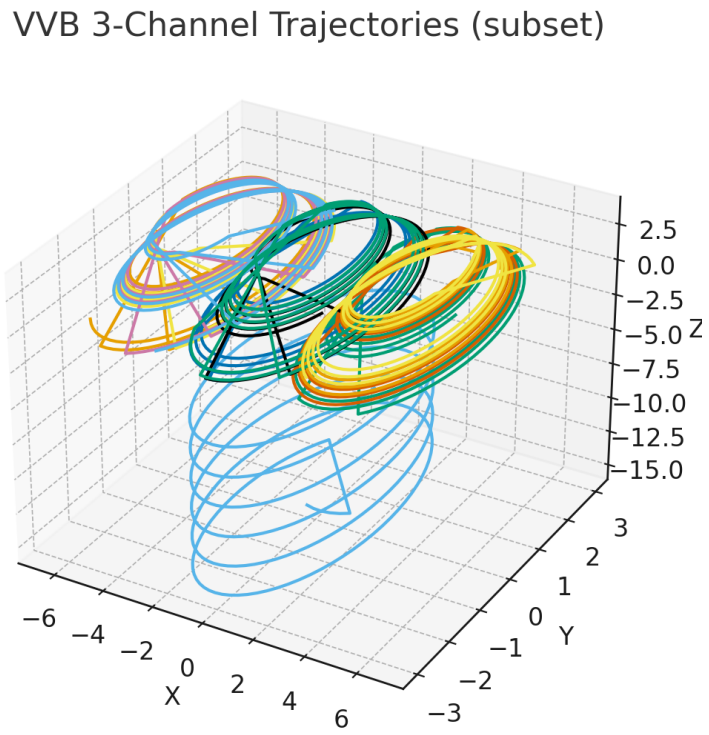


Figure 4: Simulated packet trajectories across three adjacent Vortexic Virtual Beam (VVB) channels. Colors indicate independent packet streams evolving under the Phase–Channel Allocator (PCA) policy. The helical structures exhibit smooth, low-glugging flow with occasional phase-smooth hops between channels at the predicted *kiss heights*. This visualization highlights how the VVB geometry maintains local order while evenly distributing flow across all channels, demonstrating a geometric basis for fairness and congestion relief.

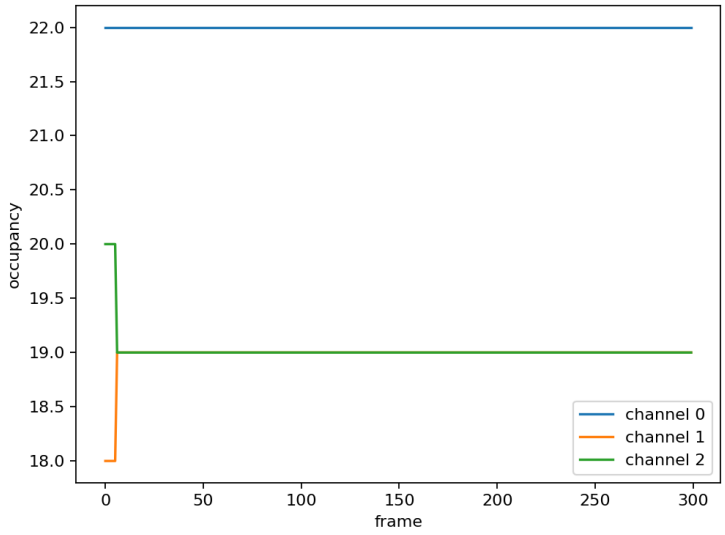


Figure 5: Per-channel occupancy over time under the Phase–Channel Allocator (PCA) policy. Lower variance across channels indicates smoother, low-glugging flow consistent with the trajectories shown in Fig. 4. Together, the two figures illustrate both the dynamic and statistical stability of multi-channel transport achieved by the Vortexic Virtual Beam geometry.

$\phi_{\min}$	$z_{\min}$	H	hops	CV	thr
0.45	0.20	20	7	0.1414	0.373
0.45	0.20	40	7	0.1414	0.373
0.45	0.20	80	7	0.1414	0.373
0.45	0.25	20	8	0.1372	0.373
0.45	0.25	40	8	0.1372	0.373
0.45	0.25	80	8	0.1372	0.373
0.45	0.35	20	13	0.1484	0.353
0.45	0.35	40	13	0.1504	0.353
0.45	0.35	80	13	0.1545	0.353
0.60	0.20	20	7	0.0707	0.373
0.60	0.20	40	7	0.0707	0.373
0.60	0.20	80	7	0.0707	0.373
0.60	0.25	20	11	0.0749	0.367
0.60	0.25	40	10	0.0998	0.367
0.60	0.25	80	10	0.0963	0.367
0.60	0.35	20	19	0.1051	0.340
0.60	0.35	40	18	0.1118	0.340
0.60	0.35	80	18	0.1009	0.340
0.75	0.20	20	9	0.1090	0.367
0.75	0.20	40	9	0.1099	0.367
0.75	0.20	80	9	0.1119	0.367
0.75	0.25	20	14	0.0595	0.360
0.75	0.25	40	13	0.0773	0.360
0.75	0.25	80	12	0.0836	0.360
0.75	0.35	20	35	0.1494	0.320
0.75	0.35	40	32	0.1572	0.320
0.75	0.35	80	30	0.1601	0.320

Table 2: Sensitivity of total hops and occupancy CV to thresholds $\phi_{\min}$, $z_{\min}$ and hysteresis H. thr = throughput per frame (stall model).

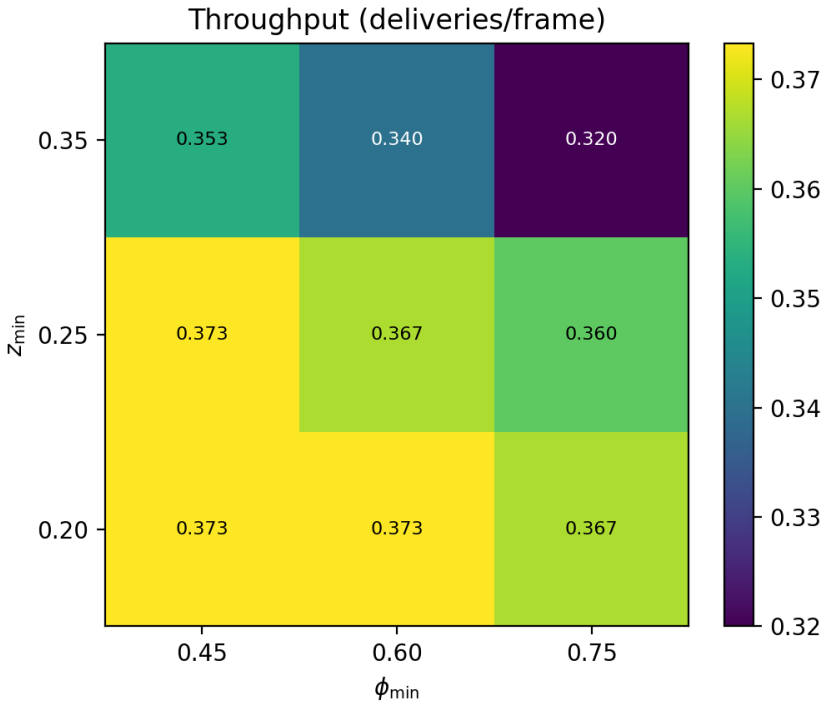


Figure 6: Throughput (deliveries/frame) as a function of thresholds $(\phi_{\min}, z_{\min})$, aggregated over $H \in \{20, 40, 80\}$. Higher is better.

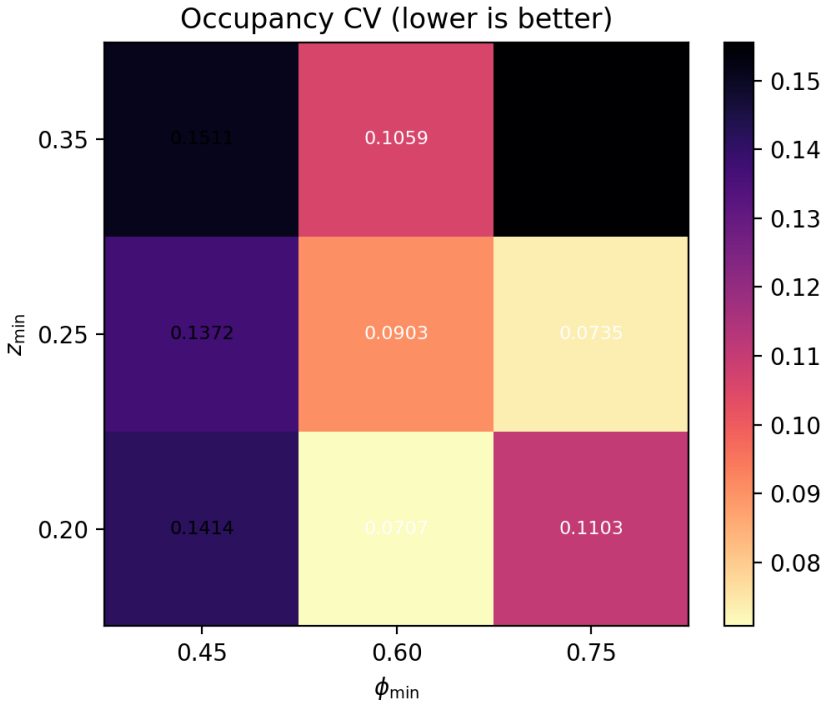


Figure 7: Occupancy coefficient of variation (CV) vs $(\phi_{\min}, z_{\min})$ (lower is better).

8 Discussion and Applications

Mapping channels to cores, queues, or memory banks suggests practical schedulers that retain ordering while relieving hot spots.

Practical reading. In our analogy, a *channel* is any service lane (core, queue, memory bank, NIC ring); ϕ tracks *local sequence/phase* (e.g., arrival order or key-range), and z tracks *pressure/age* (e.g., queue depth or wait time). Thresholds $\phi_{\min}, z_{\min}$ act like *safety spacers*: they discourage switching when it would disrupt locality, while allowing a controlled hop when a lane is hot. Hysteresis H prevents thrash.

Concrete use cases.

- **CPU/GPU work distribution.** Map channels to cores/SMs; hop long-running or contended tasks to the least-dense lane when local spacing ($\phi_{\min}, z_{\min}$) suggests a hot spot, preserving per-flow order within a lane.
- **NIC rings and message queues.** Treat RSS/NAPI rings or broker partitions as channels; hop bursts across rings to reduce head-of-line blocking without breaking per-key ordering.
- **Memory banks / HBM stacks.** View banks as channels; when row/bank conflicts accumulate (high z), hop to a less-contended bank subject to locality (ϕ) and a minimum dwell H.
- **Storage submission queues (NVMe).** Spread submissions across queues (channels) while keeping sequential streams co-located; hop on depth spikes to smooth tail latency.

Operational knobs.

- **Locality vs. relief:** smaller $\phi_{\min}$ favors locality; larger $z_{\min}$ favors hot-spot relief.
- **Stability:** larger H (hysteresis) suppresses oscillations; `phase_smooth` reduces jitter when a hop is unavoidable.
- **Policy flavor:** `prefer_kiss` biases hops to "rungs" ($\Delta z = \pi/k$) so relief steps are regular and easy to reason about.

Constraints and fit. This scheduler preserves *in-lane* order by design; it should only be applied to flows where cross-lane reordering is safe or explicitly bounded (e.g., per-key stickiness with hop only on lane boundaries). Fairness (CV) and throughput trade; operators can select a point on that curve for their SLOs.

9 Conclusion

VVB+PCA provides a minimal, geometric basis for multi-channel transport with controlled, phase-smooth hops at predictable crossover elevations. It is simple to implement and measure, and we observe reduced glugging under stress.

Appendix C: Crypto Systems Applications (Safely)

Scope. This scheduler operates at the systems layer (cores, queues, NIC rings, shards, GPUs) and does not alter cryptography or consensus rules. It preserves *in-lane* order and only moves whole flows between lanes under public, deterministic policy.

Mapping. Channels $\rightarrow$ cores/SMs, queues/partitions, NIC rings, memory banks, NVMe queues. Phase ϕ tracks locality/sequence (e.g., per-key or per-account order). Axis z tracks pressure/age (e.g., queue depth, wait time). Thresholds $(\phi_{\min}, z_{\min})$ act as safety spacers; hysteresis H prevents thrash.

Design constraints (anti-MEV / anti-reorder).

- **In-lane FIFO:** preserve per-flow order within a lane; hops occur only at flow/lane boundaries.
- **Deterministic policy:** lane choice is a pure function of (flow key, public params, current lane loads), with a hash-based tiebreak.
- **Bounded hop rate:** enforce hysteresis H and a max hops/frame to prevent oscillations or covert reordering.
- **Auditability:** log $(t, \text{flow}, \text{from} \rightarrow \text{to}, \phi, z)$ and a running hash; publish config & seeds.

Where it helps (examples).

- **ZK proving clusters (rollups):** channels=GPUs/SM groups; flows=proof-IDs. Expect steadier proofs/sec and lower P95/P99 latency.
- **Validator execution engines:** channels=execution lanes/partitions; flows=accounts/contracts. Smooth hot spots without breaking nonce order.
- **Mempool I/O:** channels=NIC rings/peer groups; flows=peer or topic. Reduce head-of-line blocking while keeping per-peer order.
- **State/storage backends:** channels=NVMe queues/shards; flows=key-ranges. Spread bursts, preserve locality.

Safe defaults (starting point). $(\phi_{\min}, z_{\min}, H) = (0.6,\ 0.25,\ 8)$, `prefer_kiss`=on, `phase_smooth`=on. Tune for your SLOs using throughput, CV, and hops/frame (report all three).

Mini-benchmark plan (protocol-agnostic).

1. Pick a target (e.g., 1–4 GPU proving node). Flows=proof-ID; channels=GPU IDs.
2. Sweep $\phi_{\min} \in [0.4, 1.0]$, $z_{\min} \in [0.15, 0.5]$, $H \in \{4, 8, 12\}$; 5 seeds.
3. Measure proofs/sec, P95/P99 latency, GPU util., CV across GPUs, hops/frame.
4. Claim: tail latency $\downarrow$ while preserving per-proof order and public determinism.

Note. This is a *scheduler*, not a consensus change. When in doubt, route conservatively: smaller $\phi_{\min}$ for locality, larger H for stability, and always publish policy/metrics.

Risk / Concern	**What it looks like**	**Mitigation (knobs / policy)**	**Monitor**
Oscillation / thrash	Frequent back-and-forth lane hops; latency variance spikes	Increase hysteresis H; enable `phase_smooth`; raise $\phi_{\min}$ slightly; cap hops per frame / per flow	Hops/frame; P95/P99 latency; hop autocorr.
Hidden reordering / MEV worries	Out-of-order events within a flow; opaque lane choices	Enforce in-lane FIFO; define explicit flow key; deterministic lane choice with hash tiebreak; append-only hop audit log	Reorder rate = 0 per flow; audit hash; config signature
Starvation / unfairness	Some lanes starved; others overloaded	Lower $z_{\min}$ (earlier relief); target least-dense lane; periodic rebalancing; cap max occupancy ratio	CV across lanes; min/median util.; backlog age
Head-of-line blocking	Bursts cause long queues in a single lane	Lower $z_{\min}$; allow burst sharding across lanes; use `prefer_kiss` to take regular relief steps	Queue-depth tails; drain time; throughput
DoS amplification / hop storms	Attack traffic triggers excessive hops and work	Rate-limit hops per flow/time; raise H under attack; admission control / priority on authenticated flows	Hop rate per flow; drops/admissions; anomaly flags
Non-determinism / disputes	Nodes disagree on lane decisions or results	Freeze policy params per release; deterministic selection; seed/tie hashes; versioned policy ID	Cross-run equality tests; policy/version hash
Locality loss / cache misses	Too many hops degrade cache/NUMA locality	Decrease $\phi_{\min}$ (favor locality); increase H; pin long-lived flows; prefer same NUMA	LLC miss rate; remote % NUMA; cache hit rate
GPU imbalance (provers)	One GPU pegged while others idle	Target *weighted* least-dense (by compute/backlog); apply recent-use penalty; smooth over k frames	GPU util. CV; SM occupancy; queue depth per GPU

Table 3: Operational risks when mapping channels to crypto systems plumbing (cores/queues/GPU rings) and mitigations using the scheduler's knobs. We recommend reporting throughput, fairness (CV), and hops/frame together.

